

Segment 2 — Face Detection & ROI Extraction

Spandan: Contactless Vital Sign Monitoring — EEE 312 DSP Project, BUET

Team Spandan

August 5, 2026

Why This Segment Exists

Every later stage of this project — filtering, CHROM/POS combination, FFT-based heart rate, and SpO2 estimation — assumes it already has three clean 1-D signals: $R(t)$, $G(t)$, $B(t)$. Those signals have to come from somewhere. This segment is that “somewhere”: it takes a raw facial video and turns it into those three traces, plus the true sampling rate f_s .

Nothing in this segment does any *signal processing* in the DSP sense yet — no filtering, no frequency analysis. It is purely computer vision (finding the face) and spatial averaging (turning a patch of pixels into one number). But if this stage picks the wrong patch of skin, or gets the sampling rate wrong, every DSP technique applied afterward is polishing a signal that was corrupted before Chapter 4 ever comes into play. Get this part right and boring; everything interesting happens later.

Intuition First

Intuition first. Think of this whole segment as building a very crude, very cheap “contact PPG sensor” out of a webcam. A real pulse oximeter clips onto a fingertip and shines light through skin, measuring how much light gets absorbed as blood volume rises and falls with each heartbeat. A camera pointed at a face is doing something similar, just passively: ambient light bounces off skin, and the *amount* of light that bounces back changes very slightly as blood volume under that skin changes. This segment’s whole job is to isolate the right patch of skin and turn its pixel brightness into a number, frame by frame, so later stages can go looking for the heartbeat hiding in that number’s tiny fluctuations.

Why Forehead Skin, Specifically

Not every visible pixel in a video frame is useful. A good ROI (region of interest) for remote photoplethysmography needs three properties:

1. **It has to be skin with real blood flow underneath it** — hair, glasses, and background contribute no physiological signal at all, only noise.
2. **It has to move as little as possible** — the mouth moves when talking, eyes move and blink, eyebrows move with expression. Any of that motion contaminates the tiny blood-flow signal with a much larger motion artifact.

3. **It has to be reasonably flat and uniformly lit** — a patch that curves sharply (like the side of the nose) picks up shading changes whenever the head turns slightly, which looks like a brightness change but has nothing to do with blood flow.

The forehead satisfies all three better than almost anywhere else on the face. It’s a broad, relatively flat, usually well-lit expanse of skin that barely moves even when a subject talks or blinks, and it sits well clear of hair (below the hairline) and well clear of eyes/eyebrows (above them). Cheeks are a common second choice in published rPPG work, but they sit closer to the mouth (jaw motion while talking or swallowing bleeds in) and are more prone to specular glare from directly overhead lighting. This project’s implementation uses a forehead-only crop for that reason — a smaller, safer target rather than trying to combine two regions and risk including the motion-heavy area between them.

Common Beginner Mistake

Common beginner mistake. It’s tempting to just average the *entire* face bounding box that the detector returns, on the theory that “more pixels means more averaging means less noise.” This is backwards. The choir-effect averaging described below only helps when every pixel is contributing the *same underlying signal* plus independent noise. Once you include eyes, eyebrows, mouth, and hair inside the average, you aren’t reducing noise anymore — you’re mixing in whole different signals (blinking, talking, hair barely reflecting light at all) that have nothing to do with blood flow and swamp out the real one. A smaller, purer patch of just forehead skin beats a bigger, messier patch of “mostly face.”

Viola-Jones Face Detection, High-Level Intuition

Finding “where is the face” in each frame is done with the Viola-Jones algorithm (`vision.CascadeObjectDetector` in MATLAB), a classical, pre-deep-learning face detector from 2001 that is still fast and reliable enough for a task like this.

The core idea is a **cascade of simple tests**. Instead of one expensive check that decides “is this a face” all at once, Viola-Jones slides a candidate window across the image (at many positions and many sizes) and runs it through a sequence of very cheap tests, each based on comparing the brightness of one rectangular region against another (for example: “the eye region is usually darker than the cheek region below it”). Any one of these tests by itself is a weak, unreliable signal — but chained together in the right order, with the cheapest and most discriminating tests first, the vast majority of non-face windows get rejected within the first one or two tests, almost for free. Only the small fraction of windows that pass *every* test in the cascade get reported as a detected face.

This is why Viola-Jones can run in real time on ordinary hardware with no GPU: it spends almost no computation on the (overwhelming majority of) regions of an image that obviously aren’t a face, and only does more work on regions that are actually promising.

Common Beginner Mistake

Common beginner mistake. It’s easy to assume a face detector either “works” or “doesn’t,” as if detection were binary and permanent per video. In practice it’s per-frame and imperfect — the exact same subject, sitting still, can have a frame where the detector confidently finds a face and the very next frame where a slight head tilt or motion blur makes it return nothing at all. Code that calls the detector once and assumes the result holds for the rest of the video

will silently break the moment that assumption fails. This segment’s implementation handles that explicitly: every detection attempt is checked for an empty result, and a fallback (the last known good box) is used instead of crashing or writing garbage.

It’s also a mistake to assume the detector only ever returns *one* box per frame just because there’s only one person in the video. Testing this implementation against a real UBFC-rPPG subject turned up exactly this: Viola-Jones returned two boxes for one frame — the real face, and a tiny false-positive box sitting on a patch of background clutter. Code that blindly takes “the first box” can just as easily lock onto the false positive as the real face, with no error or warning at all. This implementation instead compares the area of every returned box and keeps the largest one, since a real face is almost always the biggest detected region in a single-subject frontal recording. This bug was only caught because of the mandatory sanity PNG described later in this guide — the `.mat` output alone gave no indication anything was wrong.

Pause and Check Yourself

Pause and check yourself. Before reading on: why would running the full Viola-Jones cascade on *every single frame* of a ~2400-frame video be wasteful, given that the subject is sitting mostly still in front of a static camera? What would you reuse from one frame to the next instead of re-detecting from scratch?

Detecting Every Nth Frame, Not Every Frame

Running the detector on every frame is the most expensive part of this whole segment. Given that UBFC-rPPG subjects are seated and largely still, and that consecutive frames are only about 1/29th of a second apart, a face simply cannot move far enough between frames for detection to meaningfully change from one frame to the next.

This implementation exploits that: it runs the full detector once every 5 frames, and on the frames in between, reuses the most recent successfully-detected bounding box rather than re-running detection. This cuts the number of expensive detector calls by 80% for a negligible cost in ROI accuracy, for this kind of relatively static footage. The trade-off is explicit and worth stating plainly: if a subject moves their head quickly, the reused bounding box can lag behind the true face position for up to 4 frames before the next real detection call catches up. For a different dataset with faster head motion or a lower frame rate, this interval would need to be revisited — it is a tuned choice for *this* dataset’s recording conditions, not a universal constant.

Why f_s Must Be Read From the File, Never Assumed

It’s tempting to assume every video in a dataset runs at a clean 30 frames per second and hardcode that number. For UBFC-rPPG, this assumption is simply false: direct inspection of the AVI file headers in this project (see `docs/DATA_FORMAT.md`) confirms frame rates ranging from roughly 28.6 to 29.8 fps, and the exact value differs per subject.

This matters far beyond bookkeeping. Every later stage of this pipeline that involves frequency — the bandpass filter restricting the signal to the 0.7–4 Hz physiological heart-rate band, and the FFT that converts the combined pulse signal into a heart rate in bpm — depends on knowing the *true* sampling rate to correctly interpret what frequency each FFT bin corresponds to. If the sampling rate used in those calculations is off by even a few percent (30 assumed vs. 28.67 actual,

for example), every computed heart rate comes out scaled by that same few percent — a small, believable-looking error that is very easy to miss during a demo and very easy to get marked down for in a report that’s supposed to be about *classical DSP correctness*. Reading `frameRate` directly from `VideoReader`’s own header field, once per subject, costs nothing and eliminates this entire category of error.

Pause and Check Yourself

Pause and check yourself. If the true frame rate for a subject is 28.67 fps but the code assumed a flat 30 fps, and the FFT stage later detects a dominant frequency at bin corresponding to “72 bpm” under the wrong assumed rate, what would the *actual* correct heart rate be? (Hint: the error scales the same way in every frequency-domain calculation downstream of the wrong assumption.)

Spatial Averaging and the Choir Effect

Once the right patch of forehead skin is identified, the raw pixel data inside it is a small block of numbers — width $\times$ height $\times$ 3 color channels, for every single frame. The next step collapses each channel of that block down to just *one number per frame*: the mean pixel value across every pixel in the patch, done separately for red, green, and blue.

Intuition First

Intuition first — the choir effect. A single pixel’s brightness change from blood flow is minuscule — far smaller than ordinary camera sensor noise at that one pixel. Trying to read the heartbeat off of one pixel is like trying to make out a whispered word from one voice in a noisy room: technically there, practically impossible to hear cleanly. But blood volume under an entire patch of skin rises and falls *together* — every pixel in that patch carries a small piece of the same shared signal, while each pixel’s sensor noise is essentially independent and random. Averaging hundreds or thousands of pixels together is like a choir: no single voice needs to be loud or perfectly in tune, because the shared melody (the thing every voice has in common) reinforces itself across all of them, while each voice’s individual imperfections — being independent of each other — tend to cancel out rather than add up. The larger and more uniform the “choir” (the ROI), the more the shared signal stands out relative to the noise.

This is also precisely why ROI *selection* matters so much before this averaging step. The choir effect only strengthens a signal that is actually shared across the averaged pixels. Pixels from hair, background, or eyes aren’t singing the same song at all — averaging them in doesn’t add more “choir,” it adds more noise sources that have nothing in common with the blood-flow signal, which is exactly why a tight, purely-skin ROI consistently outperforms a larger but messier one.

Common Pitfalls in This Segment

Lighting changes. If ambient lighting shifts during a recording (a cloud passing outside a window, a flickering fluorescent light), that shift shows up in $R(t)$, $G(t)$, $B(t)$ as a slow drift or a sudden jump that has nothing to do with the heartbeat. This segment does not correct for it — that’s explicitly the job of the detrending/filtering stage that comes next. What this segment *is* responsible for is not making the problem worse: picking a stable, consistently-lit ROI (the forehead, rather than a

patch near a shifting shadow line) minimizes how much lighting variation gets baked into the raw signal in the first place.

Subject movement. Beyond the detection-lag trade-off already discussed, any head movement changes which pixels of the *sensor* are looking at which pixels of the *face*, even when the ROI box itself tracks correctly. Fast or frequent movement is one of the harder open problems in remote PPG generally — this classical pipeline does not attempt motion compensation beyond re-detecting the face periodically.

Detection dropouts. As covered above, Viola-Jones will occasionally fail on a perfectly normal frame. The failure mode to avoid is not the dropout itself (that’s expected and handled), but *silently* handling it in a way that corrupts the signal — for example, writing zero brightness for a dropped frame, which looks to any later stage exactly like a sudden, physiologically impossible blackout. This segment logs every frame where a fallback bounding box had to be used, both to the console and as a count saved alongside the signal, precisely so this isn’t invisible to whoever reviews the output later.

Common Beginner Mistake

Common beginner mistake. Treating a `.mat` file of *R*, *G*, *B* traces as trustworthy just because the code ran without throwing an error. A face detector can lock onto the wrong region (background texture that happens to pass the cascade, for instance) and still return a bounding box every time — no crash, no error message, just a wrong answer to every average built from then on. This is exactly why this segment mandates a sanity-check PNG for every subject: a human glancing at the image with both boxes drawn on it can catch a mis-detected or mis-cropped ROI in about two seconds, something no amount of clean code execution can guarantee on its own.

How This Segment Feeds Into the Next One

The output of this segment — `data/processed/<subjectID>_rgb_traces.mat`, containing *R*, *G*, *B*, *fs*, *subjectID*, and *numDroppedFrames* — is the entire input the next segment needs. Segment 3 (detrending and bandpass filtering, `filtering/detrendSignal.m` and `filtering/bandpassClean.m`) takes each of these three raw traces and:

1. Removes slow drift (the kind of lighting/DC-level wandering discussed above) that isn’t part of the pulsatile signal.
2. Bandpass-filters each channel to the 0.7–4 Hz range — the physiologically plausible range for human heart rate (42–240 bpm) — discarding frequency content outside that band as noise by definition.

Everything downstream of that (CHROM/POS combination, FFT heart rate, SpO2 ratio-of-ratios) assumes its input is already three reasonably clean per-channel traces sampled at a known, correct rate. That assumption is exactly what this segment is responsible for delivering — nothing more, nothing less. If a subject’s sanity PNG looks wrong, or *numDroppedFrames* is unexpectedly high for a whole recording, that is the moment to investigate — before, not after, that subject’s data is fed into three more stages of processing built on top of it.